

Regression Tree

Il seguente pdf rappresenta il report tecnico per il progetto della materia metodi avanzati di programmazione dell'anno 2025/2026 basata sugli alberi di regressione.

Introduzione al regression Tree

I Regression Tree sono una tecnica di apprendimento supervisionato utilizzata sia nell'ambito del Machine Learning sia del Data Mining, disciplina il cui obiettivo è individuare conoscenza, relazioni e pattern significativi all'interno di grandi quantità di dati. In particolare, i Regression Tree vengono impiegati per risolvere problemi di regressione, ossia problemi nei quali si desidera prevedere il valore di una variabile numerica (detta target) a partire dai valori osservati di un insieme di variabili esplicative (feature).

L'obiettivo consiste quindi nello stimare un valore continuo, come ad esempio il prezzo di un'abitazione, il consumo energetico o la previsione delle vendite di un'azienda in funzione di determinate condizioni.

Un Regression Tree viene costruito mediante un processo iterativo noto come recursive partitioning. Tale procedura suddivide ricorsivamente il dataset in sottoinsiemi sempre più omogenei rispetto alla variabile target. Ad ogni passo, l'algoritmo seleziona la variabile e la soglia di suddivisione che consentono di ridurre il più possibile l'errore di previsione, generalmente misurato attraverso la varianza o il Mean Squared Error (MSE).

Dal punto di vista strutturale, ogni nodo interno dell'albero rappresenta un test effettuato su una determinata variabile, mentre ciascun arco rappresenta l'esito di tale test. Il processo di suddivisione prosegue fino al raggiungimento di specifici criteri di arresto, come il numero minimo di osservazioni in un nodo o una profondità massima dell'albero.

I nodi foglia rappresentano invece il risultato finale della previsione: a ciascuna foglia è associato un valore numerico, generalmente ottenuto come media dei valori target delle osservazioni appartenenti a quella partizione. Il percorso dalla radice fino a una foglia identifica quindi l'insieme delle condizioni che conduce alla corrispondente previsione.

Introduzione al progetto

Il seguente progetto è composto da quattro componenti fondamentali, due riguardanti il progetto base ed altre due riguardanti le estensioni applicate dal gruppo. Le componenti sono le seguenti: - Server - Client da console - Client con GUI (acronimo di Graphical User Interface) - Client con bot Telegram

Di seguito vengono riportate singolarmente le diverse componenti.

Server

Il server si occupa della creazione del regression tree e della predizione dei valori, acquisendo i dati da un database SQL e restituendo il risultato al client.

All'avvio, il server rimane in ascolto sulla porta 8080 in attesa di una connessione da parte di un client. Una volta stabilita la connessione, il server lascia all'utente la scelta se acquisire i dati direttamente dal database oppure caricarli da un file generato in una precedente esecuzione.

Client da console

Il client da console rappresenta il client del progetto base e consente l'utilizzo completo di tutte le funzionalità offerte dal server.

All'avvio, il client richiede all'utente l'inserimento dell'indirizzo IP e del numero di porta del server. Successivamente, l'utente può scegliere se caricare i dati dal database o da un file. Gli input forniti dall'utente durante l'esecuzione influenzano direttamente il risultato elaborato dal server.

Client con GUI

Il client con GUI rappresenta una delle estensioni sviluppate dal gruppo e, analogamente al client da console, consente l'utilizzo completo di tutte le funzionalità offerte dal server.

A differenza del client da console, questa variante implementa un'interfaccia grafica realizzata con l'ecosistema JavaFX, rendendo l'esperienza d'uso più intuitiva e accessibile. L'interfaccia è suddivisa in due componenti principali:

- **Controller:** la componente scritta in Java che gestisce la logica dell'interfaccia e ne coordina il funzionamento; è la sezione scritta in java, permette all'intera interfaccia di funzionare grazie ai comandi forniti;
 - **Presentation:** la componente scritta in FXML, il linguaggio dichiarativo di JavaFX, che definisce la struttura visiva e il layout dell'interfaccia grafica.
-

Client con bot Telegram

Il client con bot Telegram rappresenta la seconda estensione sviluppata dal gruppo e offre un'alternativa al client da console attraverso l'integrazione con la piattaforma Telegram.

Il bot mette a disposizione dell'utente quattro comandi principali:

- **/start:** viene utilizzato per avviare una nuova sessione di predizione;
- **/learn:** viene utilizzato per apprendere un insieme di dati caricati dal database;
- **/load:** viene utilizzato per caricare un albero generato in una

- sessione precedente;
- /end: viene utilizzato per uscire da una predizione in corso.

Il Telegram_bot è stato implementato utilizzando la libreria **TelegramBots** di *rubenlagus* (<https://github.com/rubenlagus/TelegramBots>).

Testing

Per testare l'applicazione sono stati utilizzati unit tests realizzati tramite il framework JUnit. Inoltre non tutte le classi sono state testate, in seguito saranno riportate le classi non testate e la motivazione:

Package data

- Attribute: è nella lista delle classi non testate poichè è una classe astratta. La classe Attribute definisce una struttura comune alle sottoclassi ed il comportamento delle implementazioni verrà testato nelle rispettive classi dei test;
- ContinuousAttribute: è nella lista delle classi non testate poichè è una classe che non aggiunge nuova logica. Non aggiunge campi, metodi e non effettua override;
- TrainingDataException: è nella lista delle classi non testate poichè rappresenta solamente un'estensione dell'eccezione Exception.

percorso: *progetto_map/project/mapServer/src/data/*

Package database

- DatabaseConnectionException: è nella lista delle classi non testate poichè rappresenta solamente un'estensione dell'eccezione Exception;
- EmptySetException: è nella lista delle classi non testate poichè rappresenta solamente un'estensione dell'eccezione Exception.

percorso: *progetto_map/project/mapServer/src/database/*

Package application

- Load_From_Controller: è nella lista delle classi non testate poichè testando questa classe si testerebbe la logica di JavFX, ma non è una nostra responsabilità;
- Controller: è nella lista delle classi non testate poichè testando questa classe si testerebbe la logica di JavFX, ma non è una nostra responsabilità. Ad eccezion fatta per il metodo validate, che implementa una logica;
- RtClient: è nella lista delle classi non testate poichè questa classe viene testata indirettamente testando ServerOneClient;

- `ServerException`: è nella lista delle classi non testate poichè rappresenta solamente un'estensione dell'eccezione `Exception`.

percorso: *progetto_map/estensioni/GUI/mapClient/application/*

Package Presentation

Tutti i metodi presenti in questo package non sono stati testati, poichè rappresentano delle classi di sola stampa. **percorso:**
progetto_map/estensioni/GUI/mapClient/Presentation/

Package telegram_bot/ mapClient

- `Command`: è nella lista delle classi non testate poichè non implementa logica, ma solo l'implementazione dei comandi del bot telegram;
- `MapBot`: è nella lista delle cose non testate poichè testando questa classe si testerebbe la logica della libreria telegrambots, ma non è una nostra responsabilità;
- `ServerConnection`: è nella lista delle classi non testate poichè questa classe viene testata indirettamente testando `ServerOneClient`.

n.b.: è stato specificato "telegram_bot/mapClient" come percorso poichè è il `mapClient`. Specificando solo il nome del package "mapClient" ci sarebbero potute essere ambiguità.

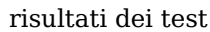
percorso:
progetto_map/estensioni/telegram_bot/mapClient/src/mapClient/

Package utility

- `Keyboard`: è nella lista delle classi non testate poichè non è una classe creata da noi.

percorso: *progetto_map/project/mapClient/src/utility/*

In seguito verranno riportati i risultati dei test, con una percentuale di coverage del 78.1%:



I due test `constructorThrowsForEmptyTable` e `constructorThrowsIfLastColumnNotNumeric` sono marcati con l'annotazione `@Disabled` in quanto verificano due specifici percorsi di errore del costruttore di `Data` che richiedono la presenza nel database di tabelle con caratteristiche particolari: rispettivamente una tabella con schema valido ma priva di righe e una tabella la cui ultima colonna sia di tipo stringa. Poiché tali tabelle non sono disponibili nel database di test utilizzato, eseguire questi test produrrebbe un fallimento non dovuto a un difetto del codice ma alla mancanza dell'infrastruttura necessaria. Si è scelto `@Disabled` invece di `Assumptions` perché la condizione che impedisce l'esecuzione non è dinamica — come potrebbe essere la raggiungibilità del database — ma è nota staticamente prima ancora di avviare i test. Il messaggio descrittivo allegato all'annotazione documenta esplicitamente la lacuna e indica le azioni necessarie per colmarla: creare nel database le tabelle `empty_table` e `string_last_col_table` con le caratteristiche richieste, dopodiché sarà sufficiente rimuovere l'annotazione `@Disabled` per rendere i test operativi.